When you receive a packet, it will look like what's shown is Figures 9 and 10.

Sending packets with a raw socket

To send a packet, we first have to know the source and destination IP addresses as well as the MAC address. Use your friend's *MAC & IP* address as the destination IP and MAC address. There are two ways to find out your IP address and MAC address:

1. Enter *ifconfig* and get the IP and MAC for a particular interface.
2. Enter *ioctl* and get the IP and MAC.

The second way is more efficient and will make your program machine-independent, which means you should not enter *ifconfig* in each machine.

Opening a raw socket

To open a raw socket, you have to know three fields of socket API -- *Family- AF_PACKET, Type- SOCK_RAW* and for the protocol, let's use *IPPROTO_RAW* because we are trying to send an IP packet. *IPPROTO_RAW* macro is defined in the *in.h* header file:

```
sock_raw=socket(AF_PACKET,SOCK_RAW,IPPROTO_RAW);
if(sock_raw == -1)
     printf("error in socket");
```

What is *struct ifreq*?

Linux supports some standard *ioctls* to configure network devices. They can be used on any socket's file descriptor, regardless of the family or type. They pass an *ifreq* structure, which means that if you want to know some information about the network, like the interface index or interface name, you can use *ioctl* and it will fill the value of the *ifreq* structure passed as a third argument. In short, the *ifreq* structure is a way to get and set the network configuration. It is defined in the *if.h* header file or you can check the man page of *netdevice* (see Figure 11).

Getting the index of the interface to send a packet

There may be various interfaces in your machine like *loopback, wired interface* and *wireless interface*. So you have to decide the interface through which we can send our packet. After deciding on the interface, you have to get the index of that interface. For this, first give the name of the interface by setting the field *ifr_name* of *ifreq* structure, and then use *ioctl*. Then use the *SIOCGIFINDEX* macro defined in *sockios.h* and you will receive the index number in the *ifreq* structure:

```
struct ifreq ifreq_i;
memset(&ifreq_i,0,sizeof(ifreq_i));
strncpy(ifreq_i.ifr_name,"wlan0",IFNAMSIZ-1); //giving name
of Interface
if((ioctl(sock_raw,SIOCGIFINDEX,&ifreq_i))<0)
     printf("error in index ioctl reading");//getting Index
Name

printf("index=%d\n",ifreq_i.ifr_ifindex);
```

Getting the MAC address of the interface

Similarly, you can get the MAC address of the interface, for which you need to use the *SIOCGIFHWADDR* macro to *ioctl*:

```
struct ifreq ifreq_c;
memset(&ifreq_c,0,sizeof(ifreq_c));
strncpy(ifreq_c.ifr_name,"wlan0",IFNAMSIZ-1);//giving name of
Interface

if((ioctl(sock_raw,SIOCGIFHWADDR,&ifreq_c))<0) //getting MAC
Address
     printf("error in SIOCGIFHWADDR ioctl reading");
```

Getting the IP address of the interface

For this, use the SIOCGIFADDR macro:

```
struct ifreq ifreq_ip;
memset(&ifreq_ip,0,sizeof(ifreq_ip));
strncpy(ifreq_ip.ifr_name,"wlan0",IFNAMSIZ-1);//giving name
of Interface
if(ioctl(sock_raw,SIOCGIFADDR,&ifreq_ip)<0) //getting IP
Address
{
printf("error in SIOCGIFADDR \n");
}
```

Constructing the Ethernet header

After getting the index, as well as the MAC and IP addresses of an interface, it's time to construct the Ethernet header. First, take a buffer in which you will place all information like the Ethernet header, IP header, UDP header and data. That buffer will be your packet.

```
sendbuff=(unsigned char*)malloc(64); // increase in case of
more data
memset(sendbuff,0,64);
```

To construct the Ethernet header, fill all the fields of the *ethhdr* structure:

```
struct ethhdr *eth = (struct ethhdr *)(sendbuff);

eth->h_source[0] = (unsigned char)(ifreq_c.ifr_hwaddr.sa_
data[0]);
eth->h_source[1] = (unsigned char)(ifreq_c.ifr_hwaddr.sa_
data[1]);
eth->h_source[2] = (unsigned char)(ifreq_c.ifr_hwaddr.sa_
data[2]);
```